COMP0008:
Computer Architecture and Concurrency

Live Session (week 8)

Stefano Vissicchio

References: GPBBHL chapter 2

Warmup

- Self-assess on www.menti.com w/ code 3887 2436
  - would help me try and tune review speed

- Interactions have been great so far
  - let's keep going!

- Other experiment: a few more Mentimeter polls *during* the session (e.g., on in-class questions)
  - it'd give all of you the opportunity to stop and think about key points before we discuss altogether
  - .. but let me know if you don't like the idea!

The concurrency story so far

- Modern hardware offers support to true parallelism, enabling better performance
  - threads can be executed simultaneously on different cores

The concurrency story so far

- Modern hardware offers support to true parallelism, enabling better performance
  - threads can be executed simultaneously on different cores
- Threads' interactions can non-deterministically lead to unwanted results
  - depending on hardware, OS, workload, interrupts, etc.
  - interactions can be modeled by concurrency abstraction

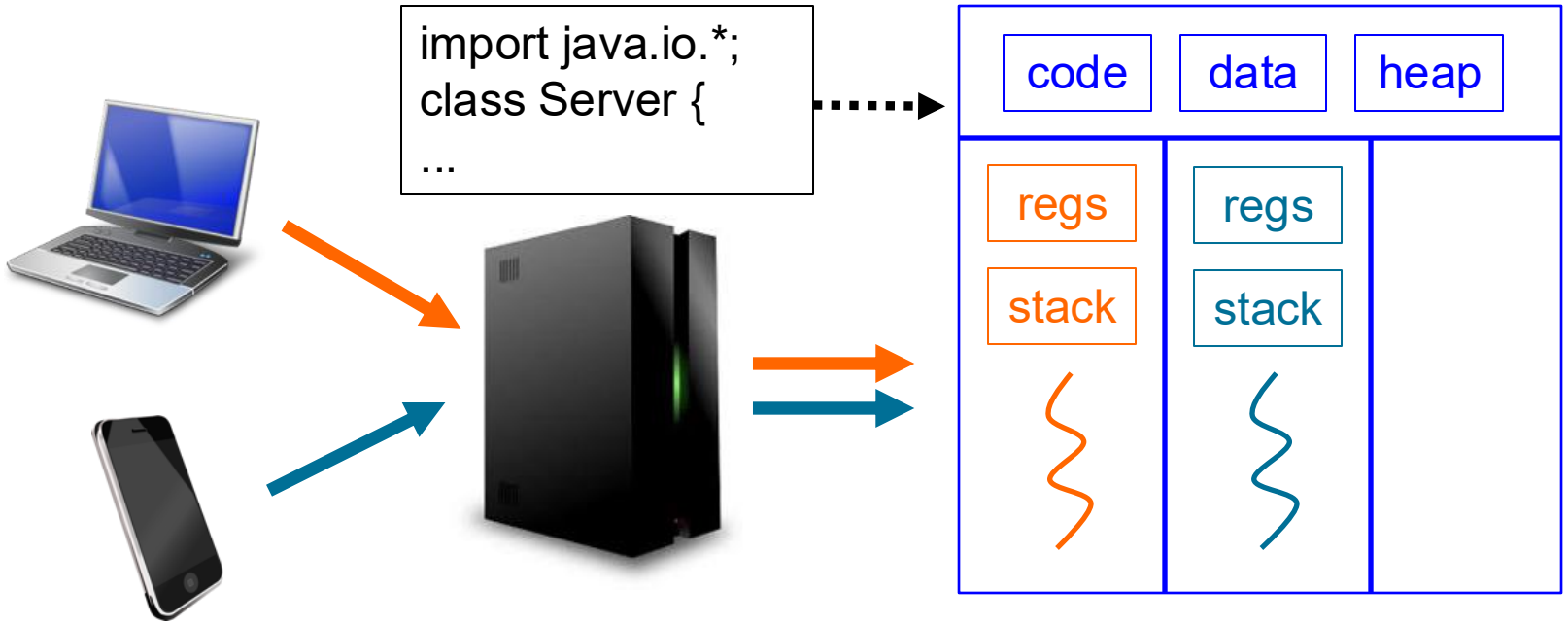
The concurrency story so far

- Modern hardware offers support to true parallelism, enabling better performance
  - threads can be executed simultaneously on different cores
- Threads' interactions can non-deterministically lead to unwanted results
  - depending on hardware, OS, workload, interrupts, etc.
  - interactions can be modeled by concurrency abstraction
- We generally need to forbid some interleavings to guarantee correctness

- Let's now see some Java code!
  - making the above points more concrete

Let's reconsider a simple Web service

- Focus on a server that factorizes input numbers
  - each client's request contains an integer X; the server responds to it with a list of factors that multiply to X
- We want to assign one thread per client request
  - for server performance and reactiveness



Let's rely on Java Servlets

- Servlets are a framework to implement Java programs that run within a Web server
  - provide native support for request-response applications
- To define our own application, we need to implement new servlet classes
  - each servlet defines a component of the server application
  - a new servlet can be defined by implementing methods specified by the Servlet interface

Let's rely on Java Servlets

- Servlets are a framework to implement Java programs that run within a Web server
  - provide native support for request-response applications
- To define our own application, we need to implement new servlet classes
  - each servlet defines a component of the server application
  - a new servlet can be defined by implementing methods specified by the Servlet interface
- Concurrency model:
  - each client request is served by one thread
  - each servlet can be called from multiple threads
  - threads accessing same servlet share the servlet's state

Java Servlets in this module

- Goals:
  - demonstrate that concurrency is crucial even “just” to use (some) existing frameworks or libraries
  - possibly, help you with background, content and examples provided in [GPBBHL]
- Non-goals:
  - you do ***not*** need to study servlets in depth, ***nor*** to check all classes and methods in the servlet package
- More resources (if needed): Java documentation
  - e.g., https://docs.oracle.com/javaee/5/tutorial/doc/bnafe.html

Iteration 1: SimpleFactorizer

- We first implement the basic service as follows:

```
public class SimpleFactorizer implements Servlet
{
  public void service(ServletRequest req,
                      ServletResponse resp) {
    BigInteger i = extract(req);
    BigInteger[] factors = factorize(i);
    encodeInResponse(resp, factors);
  }
  ...
}
```

Is SimpleFactorizer correct?

- We first implement the basic service as follows:

```
public class SimpleFactorizer implements Servlet
{
  public void service(ServletRequest req,
                      ServletResponse resp) {
    BigInteger i = extract(req);
    BigInteger[] factors = factorize(i);
    encodeInResponse(resp, factors);
  }
  ...
}
```

- We need to define what correctness means for a multi-threaded application!

Defining correctness in multi-threaded world

- Thread safety: correctness (never in an invalid state) irrespectively of the interactions between threads
  - correctness can be formalized as pre-conditions, post-conditions and invariants
- Thread safety implies no race condition
  - race condition: incorrect computations in specific interleavings (e.g., unlucky timings)

Defining correctness in multi-threaded world

- Thread safety: correctness (never in an invalid state) irrespectively of the interactions between threads
  - correctness can be formalized as pre-conditions, post-conditions and invariants
- Thread safety implies no race condition
  - race condition: incorrect computations in specific interleavings (e.g., unlucky timings)
- In this module, we will often adopt a practical definition of thread safety
  - generally, we say that an application (resp. class) is thread safe if it allows the same set of outputs (resp. states) regardless of number and timings of threads

Defining correctness in multi-threaded world

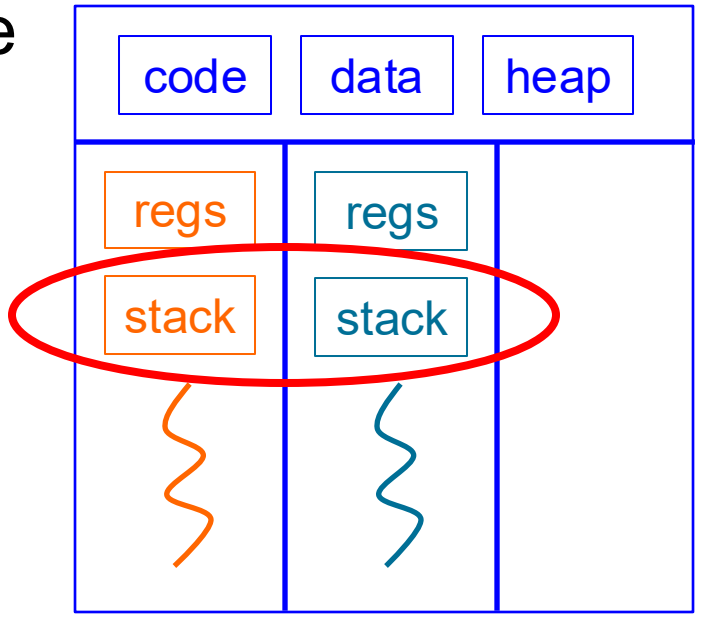
- Thread safety: correctness (never in an invalid state) irrespectively of the interactions between threads
  - correctness can be formalized as pre-conditions, post-conditions and invariants
- Thread safety implies no race condition
  - race condition: incorrect computations in specific interleavings (e.g., unlucky timings)

- Back to question: Is `SimpleFactorizer` thread safe?
  - motivate your answer

SimpleFactorizer is thread safe

- Thread safety: correctness (never in invalid state) irrespectively of the interactions between threads
  - correctness can be formalized as pre-/post-conditions and invariants; implies no race conditions

- `SimpleFactorizer` is thread safe
  - when calling the `service()` method, any thread stores variables local to the method in its own stack, which is not accessible by other threads



Iteration 2: adding application-level caching

- Let's now try and improve the server's performance
  - factorizing is an expensive operation
  - we'd like to avoid factorizing a number N if we have factorized it recently
- We start by remembering the last factorization
  - can be generalized later by remembering the last N numbers or the K most frequently requested ones
- To do so, we add instance fields to our servlet

```
public class CachingFactorizer implements Servlet {
  private BigInteger lastNumber;
  private BigInteger[] lastFactors;
  ...
```

Iteration 2: adding application-level caching

```
  ...
  public void service(ServletRequest req,
                      ServletResponse resp) {
    BigInteger i = extract(req);
    BigInteger[] factors = null;
    if(i.equals(lastNumber)){
      factors = lastFactors.clone();
    }
    if(factors == null){
      factors = factorize(i);
      lastNumber = i;
      lastFactors = factors;
    }
    encodeInResponse(resp, factors);
  }
}
```

Don't need to factorize

Need to factorize, then update instance fields

Is CachingFactorizer thread safe?

```
  ...
  public void service(ServletRequest req,
                      ServletResponse resp) {
    BigInteger i = extract(req);
    BigInteger[] factors = null;
    if(i.equals(lastNumber)){
      factors = lastFactors.clone();
    }
    if(factors == null){
      factors = factorize(i);
      lastNumber = i;
      lastFactors = factors;
    }
    encodeInResponse(resp, factors);
  }
}
```

This is **not** thread safe!

```
...
public void service(ServletRequest req,
                    ServletResponse resp) {
  BigInteger i = extract(req);
  BigInteger[] factors = null;
  if(i.equals(lastNumber)){
    factors = lastFactors.clone();
  }
  if(factors == null){
    factors = factorize(i);
    lastNumber = i;
    lastFactors = factors;
  }
  encodeInResponse(resp, factors);
}
}
```

Post-condition #1: the product of lastFactors equals lastNumber

Post-condition #2: each response encodes the factors for the number in the corresponding request

Can you spot violations of any post-condition?

Example race condition

```
  ...
  public void service(ServletRequest req,
                      ServletResponse resp) {
    BigInteger i = extract(req);
    BigInteger[] factors = null;
    if(i.equals(lastNumber)){
      factors = lastFactors.clone();
    }
    if(factors == null){
      factors = factorize(i);
      lastNumber = i;
      lastFactors = factors;
    }
    encodeInResponse(resp, factors);
  }
}
```

Two threads A and B:
[A] lastNumber = X
[B] lastNumber = Y
[B] lastFactors = factorize(Y)
[A] lastFactors = factorize(X)

Outcome:
lastNumber = Y
lastFactors = factorize(X)

Another race condition

```
  ...
  public void service(ServletRequest req,
                      ServletResponse resp) {
    BigInteger i = extract(req);
    BigInteger[] factors = null;
    if(i.equals(lastNumber)){
      factors = lastFactors.clone();
    }
    if(factors == null){
      factors = factorize(i);
      lastNumber = i;
      lastFactors = factors;
    }
    encodeInResponse(resp, factors);
  }
}
```

Two threads A and B:
[A] lastNumber.equals(X)
[B] lastFactors = factorize(Y)
[A] factors = lastFactors

Outcome: A sends back the factors of number Y (handled by B) instead of those for the input X

Analysis: we need synchronization!

- Key observation: Multiple threads have access to the same variables
  - lastNumber and lastFactors are instance fields, and hence they are shared across threads
- Problem: Shared variables are modified by different threads that run concurrently
  - inconsistent modifications of lastNumber and lastFactors

Analysis: we need synchronization!

- Key observation: Multiple threads have access to the same variables
  - lastNumber and lastFactors are instance fields, and hence they are shared across threads
- Problem: Shared variables are modified by different threads that run concurrently
  - inconsistent modifications of lastNumber and lastFactors
- Solution: We must coordinate threads by enforcing atomicity of actions on shared variables
  - this will restrict the set of possible interleavings by synchronizing access to shared variables
  - sequences of operations that must be atomic to ensure thread safety are called compound actions

Iteration 3: adding synchronization

```
  ...
  public synchronized void service(ServletRequest req,
                                   ServletResponse resp) {
    BigInteger i = extract(req);
    BigInteger[] factors = null;
    if(i.equals(lastNumber)){
      factors = lastFactors.clone();
    }
    if(factors == null){
      factors = factorize(i);
      lastNumber = i;
      lastFactors = factors;
    }
    encodeInResponse(resp, factors);
  }
}
```

Only one thread at the time can execute this method.

Technically, one thread acquires a lock (linked to this object) before executing this method: other threads trying to execute this method are forced to wait for the lock to be released.

Are we happy now?

```
  ...
  public synchronized void service(ServletRequest req,
                                   ServletResponse resp) {
    BigInteger i = extract(req);
    BigInteger[] factors = null;
    if(i.equals(lastNumber)){
      factors = lastFactors.clone();
    }
    if(factors == null){
      factors = factorize(i);
      lastNumber = i;
      lastFactors = factors;
    }
    encodeInResponse(resp, factors);
  }
}
```

Only one thread at the time can execute this method.

Technically, one thread acquires a lock (linked to this object) before executing this method: other threads trying to execute this method are forced to wait for the lock to be released.

This is **slow**!

```
  ...
  public synchronized void service(ServletRequest req,
                                   ServletResponse resp) {
    BigInteger i = extract(req);
    BigInteger[] factors = null;
    if(i.equals(lastNumber)){
      factors = lastFactors.clone();
    }
    if(factors == null){
      factors = factorize(i);
      lastNumber = i;
      lastFactors = factors;
    }
    encodeInResponse(resp, factors);
  }
}
```

Only one thread at the time performs the expensive operation.

Outcome: despite we used multi-threading *and* caching, performance is very close to single-threaded application!

Iteration 4: better locking

```
public void service(ServletRequest req,
                    ServletResponse resp) {
  BigInteger i = extract(req);
  BigInteger[] factors = null;
  synchronized(this){
    if(i.equals(lastNumber))
      factors = lastFactors.clone();
  }
  if(factors == null){
    factors = factorize(i);
    synchronized(this) {
      lastNumber = i; lastFactors = factors;
    }
  }
  encodeInResponse(resp, factors);
}
```

Only reads and updates to lastNumber and lastFactors are serialized.

Multiple threads can factorize integers in parallel.

Iteration 5: possible refactoring

- We can finally isolate critical sections of the code in synchronized methods

```
...
public void service(ServletRequest req,
                    ServletResponse resp) {
  BigInteger i = extract(req);
  BigInteger[] factors = getSavedFactors(i);
  if(factors == null){
    factors = factorize(i);
    saveState(i, factors);
  }
  encodeInResponse(resp, factors);
}
...
```

Iteration 5: possible refactoring

```
  ...
  private synchronized BigInteger[]
                              getSavedFactors(BigInteger i) {
    if(i.equals(lastNumber)){
      return lastFactors.clone();
    }
    return null;
  }
  ...
  private synchronized void saveState(BigInteger i,
                                      BigInteger[] factors) {
    lastNumber = i;
    lastFactors = factors;
  }
}
```

Lessons (to be) learned: centrality of state

- Object's state: internal data that affect the object's externally visible behaviour
  - typically, instance and static fields
  - can also include fields from dependent objects
- Threads that share and arbitrarily modify state can cause concurrency problems, called interference
  - e.g., state inconsistently mixes updates from two threads

Lessons (to be) learned: centrality of state

- Object's state: internal data that affect the object's externally visible behaviour
  - typically, instance and static fields
  - can also include fields from dependent objects
- Threads that share and arbitrarily modify state can cause concurrency problems, called interference
  - e.g., state inconsistently mixes updates from two threads
- We **must** synchronize threads' access to shared mutable state; otherwise, the program is broken
  - general rule: whenever multiple threads access a given mutable state variable, we must coordinate their access to it using synchronization

Lessons (to be) learned: importance of design

- Problem: retrofitting thread-safety is hard
  - it entails evaluating *all possible accesses* to any shared resources by any set of spawn threads; so, it typically requires checking very many code paths and interleavings
- Solution: design with thread-safety in mind
  - for each state variable, consider if: (i) it needs to be shared, (ii) it must be mutable, (iii) access to it must be synchronized

Lessons (to be) learned: importance of design

- Problem: retrofitting thread-safety is hard
  - it entails evaluating *all possible accesses* to any shared resources by any set of spawn threads; so, it typically requires checking very many code paths and interleavings
- Solution: design with thread-safety in mind
  - for each state variable, consider if: (i) it needs to be shared, (ii) it must be mutable, (iii) access to it must be synchronized
- *Reminder*: we often face a tradeoff between correctness, simplicity and performance
  - e.g., using synchronization everywhere makes it simpler to write multi-threaded code but can hurt performance... and even correctness! [week 10]

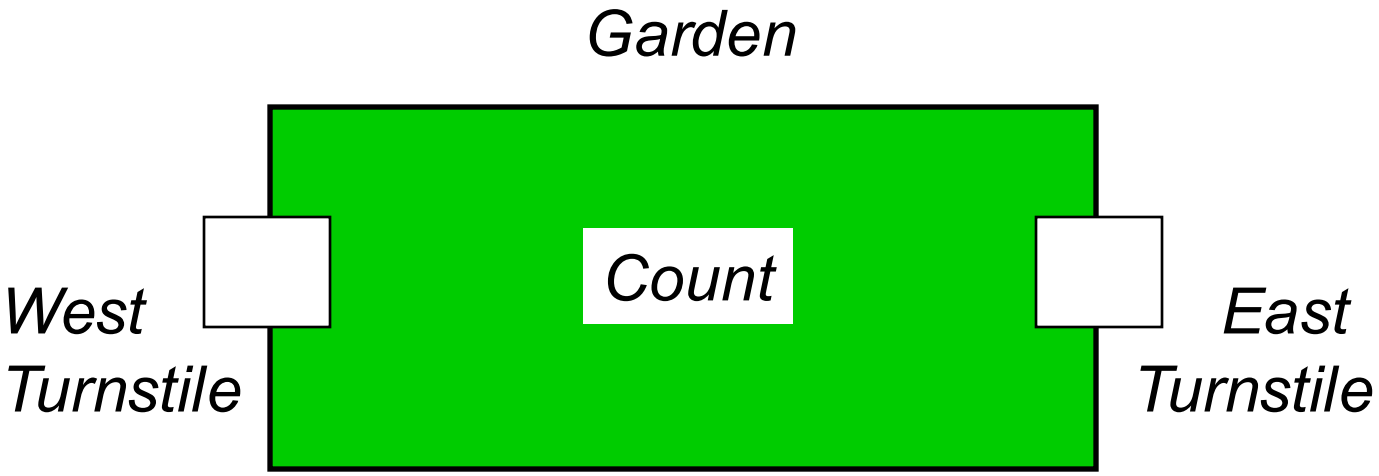
Lessons (to be) learned: available tools

- Locking: the `synchronize` keyword enables use of Java implicit locks that enforce mutual exclusion
  - blocks of instructions guarded by the same lock cannot be executed by more than one thread, at any time
  - basis for supporting synchronization policies

- Good software engineering practices useful here too
  - encapsulation and data hiding: internal state of an object is directly accessible *only by the object itself*
  - immutability: make variable immutable when their value should not change over time
  - documentation, especially of invariants

The Big Garden application: overview

- Inspired by Ornamental Garden [Magee & Kramer ch.4]
  - Garden open to the public, with people entering through either one of two turnstiles
  - Visitors counted at each turnstile and for the entire garden

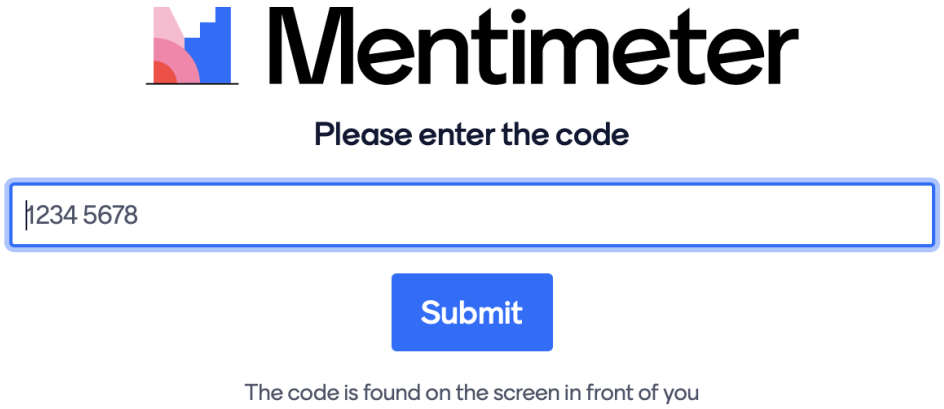


©Magee/Kramer

- Big Garden is a generalization for N turnstiles
  - … with simplified and updated code

Let's test our understanding: CW5

- Please, connect to www.menti.com
- There, enter code 3887 2436



Let's test our understanding

- Can two threads run b() and c() concurrently on the same Example instance?

```
public class Example {
  private int x = 2;
  private int y = 3;

  public synchronized void b() {
    x = x + y;
  }

  public synchronized void c() {
    y = 2 * x;
  }
  ...
}
```

Let's test our understanding

- Can two threads run b() and c() concurrently?

  Not on the same instance, by def. of synchronized

```
public class Example {
  private int x = 2;
  private int y = 3;

  public synchronized void b() {
    x = x + y;
  }

  public synchronized void c() {
    y = 2 * x;
  }
  ...
}
```

Let's test our understanding

- What are the possible values of [x,y] if b() and c() are run on the same Example instance?

```
public class Example {
  private int x = 2;
  private int y = 3;

  public synchronized void b() {
    x = x + y;
  }

  public synchronized void c() {
    y = 2 * x;
  }
  ...
}
```

Let's test our understanding

- What are the possible values of [x,y] if b() and c() are run on the same Example instance?

```
public class Example {
  private int x = 2;
  private int y = 3;

  public synchronized void b() {
    x = x + y;
  }

  public synchronized void c() {
    y = 2 * x;
  }
  ...
}
```

Only two possible interleavings

Interleaving #1:
First b() → [5,3]
Then c() → [5,10]

Interleaving #2:
First c() → [2,4]
Then b() → [6,4]

Let's test our understanding

- Can two threads run a() and b() concurrently on the same Example instance?

```
public class Example {
  private int x = 2;
  private int y = 3;

  public void a() {
    y = 2 * x;
  }

  public synchronized void b() {
    x = x + y;
  }
  ...
}
```

Let's test our understanding

- Can two threads run a() and b() concurrently?

  Yes, even on same instance: a() is not synchronized

```
public class Example {
  private int x = 2;
  private int y = 3;

  public void a() {
    y = 2 * x;
  }

  public synchronized void b() {
    x = x + y;
  }
  ...
}
```

Let's test our understanding

- What are the possible values of [x,y] if a() and b() are run concurrently on the same instance?

```
public class Example {
  private int x = 2;
  private int y = 3;

  public void a() {
    y = 2 * x;
  }

  public synchronized void b() {
    x = x + y;
  }
  ...
}
```

Let's test our understanding

- What are the possible values of [x,y] if a() and b() are run concurrently on the same instance?

```
public class Example {
  private int x = 2;
  private int y = 3;

  public void a() {
    y = 2 * x;
  }

  public synchronized void b() {
    x = x + y;
  }
  ...
}
```

Effectively, no synchronization → all possible interleavings of atomic actions

Outcome: [5,4] is also possible

E.g., interleaving #3:
a() reads x = 2
b() sets x = 5
a() sets y = 2 * 2 = 4

Let's test our understanding

- Is the Example class thread safe?

```
public class Example {
  private int x = 2;
  private int y = 3;

  public void a() {
    y = 2 * x;
  }

  public synchronized void b() {
    x = x + y;
  }
  ...
}
```

Let's test our understanding

- Is the Example class thread safe?

```
public class Example {
  private int x = 2;
  private int y = 3;

  public void a() {
    y = 2 * x;
  }

  public synchronized void b() {
    x = x + y;
  }
  ...
}
```

Technically, depends on the specifications.

Note however that threads can interfere when running a(), and generate values of [x,y] impossible otherwise: so, in practice, we can say that the class is not thread safe.

Additional content

- Specifics about Java threads [Moodle]
  - including Java syntax and threads' lifecycle
- Standalone Java code [Moodle]
  - not relying on external frameworks like the Servlet one
- Use of thread-safe libraries [GPBBHL ch. 2]
  - e.g., those in `java.util.concurrent` package
  - ***main takeaway***: thread-safe libraries are useful but do NOT solve all concurrency problems in your application!
- Lock reentrancy [GPBBHL 2.3.2]
  - threads can re-acquire locks they already hold